

Prezentare generală

În cadrul temei, vom dezvolta și implementa un protocol orientat pe conexiune peste UDP, care permite utilizatorului să trimită un flux de octeți fără erori între două dispozitive pe care le vom numi client și server. **Vom folosi acest protocol pentru a transmite imagini și cărți.** În acest scenariu, clientul doar va trimite date, iar serverul le va recepționa și va trimite segmente de control pentru a-l informa de primirea corectă a acestora.

Pentru protocolul nostru, unitatea de transmisie se numește segment. Segmentele le vom transmite prin intermediul protocolului UDP.

Protocolul dezvoltat **trebuie** să asigure transferul corect de fișiere peste o legătura de date care are următoarele caracteristici:

- Are o rata nespecificată de pierdere a segmentelor
- are o viteză de transmisie
- introduce o latență constantă a pachetelor

Legătura de date trebuie să fie folosită optim de către protocol în funcție de latență și viteză de transmisie. Implementarea pe care o veți face va avea o fereastră glisantă atât la sender, cât și la receiver.

Spre deosebire de tema precedentă, unde am implementat o specificație dată, în această temă, trebuie să definiți și să implementați propriul vostru protocol. Vă recomandăm să gândiți specificația protocolului pe foaie înainte de a vă apuca de implementare.

Lectură temă

- [TCP: Transmission control protocol](#) (8 min)
- [TCP connection walkthrough](#) 9 min
- [Go-back-n and selective repeat](#)
- [TCP Flow Control](#)
- [The Transmission Control Protocol](#)

Protocol

Protocolul trebuie să aibă următorul header pentru date:

```
|Protocol ID|Connection ID|Type|SEQ NUM|Payload Size|Payload|
```

Segmentele de tip control vor avea următoarea structură:

```
|Protocol ID|Connection ID|Type|SEQ NUM|ACK NUM|RECV WINDOW SIZE|
```

Nu vom include un checksum în protocolul nostru deoarece avem deja implementarea din UDP care va face drop la datagramele corupte.

Dacă aveți nevoie, puteți extinde header-ul cu alte câmpuri, dar trebuie să păstrați câmpurile existente în ordinea dată.

Campurile din protocol au urmatoarele functionalitati:

- Protocol ID - Identificator al protocolului nostru pe 2 bytes
- Connection ID - Identificator al conexiunii
- Seq Number - Numărul de secvență al segmentului
- Type - Tipul pachetului (e.g. Data, ACK)
- ACK Num - Folosit de receiver pentru a indica un număr de secvență acceptat. Pentru performanță, se folosește tehnica de **Cumulative ACK (ACK cumulativ)**, care presupune trimiterea unui singur ACK care confirmă primirea tuturor segmentelor până la un anumit număr de secvență, în loc să trimită un ACK separat pentru fiecare segment primit.
- Recv Window Size - Este folosit de receiver pentru a indica cât spațiu are disponibil în buffer-ul intern. Dacă, de exemplu, receptorul nu mai are spațiu de stocare pentru segmente, va trimite un segment de control cu `recv_window_size` 0 pentru a înștiința transmitătorul de acest lucru. Această tehnică o găsim sub denumirea de **Flow Control**.

Vom folosi următoarele structuri pentru a reprezenta cele două tipuri de segmente:

```
/* Header segment Data. */
struct __attribute__((packed)) poli_tcp_data_hdr {
    uint8_t protocol_id;
    uint8_t conn_id;
    uint8_t type;
    uint16_t seq_num;
    uint16_t len;
};

/* Header segment control */
struct __attribute__((packed)) poli_tcp_ctrl_hdr {
    uint8_t protocol_id;
    uint8_t conn_id;
    uint8_t type;
    uint16_t ack_num;
    uint16_t recv_window;
};
```

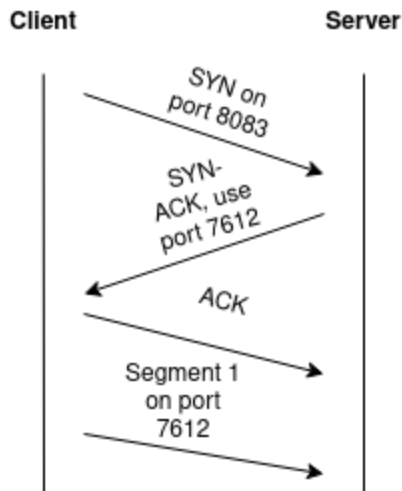
Vom folosi valoarea 42 pentru câmpul `Protocol ID`. Valorile folosite în celelalte câmpuri sunt la latitudinea voastră.

```
#define POLI_PROTOCOL_ID 42
```

Stabilirea Conexiunii

Protocolul va folosi tehnica de three way handshake pentru a stabili o conexiune. În această etapă se poate stabili valoarea inițială a numărului de secvență, ID-ului de conexiune și receiver-ul informează sender-ul de dimensiunea maximă a ferestrei de recepționare.

Vom folosi portul 8032 pe server pentru a accepta conexiuni. Segmentul SYN-ACK va fi un segment special de control, va avea un payload ce conține portul pe care va avea loc comunicarea cu acel client. Portul este ales aleator. Clientul va trimite fișierul pe acest port.



ACK-ul poate fi trimis pe oricare dintre porturi, nu există o restricție în acest sens.

O să avem astfel un singur socket folosit de către client atât pentru three-way handshake, cât și pentru date. În schimb, la receptor vom avea un socket dedicat primirii de conexiuni și un socket pentru fiecare client. Aceasta funcționalitate este similară cu **accept**.

Selective repeat ARQ

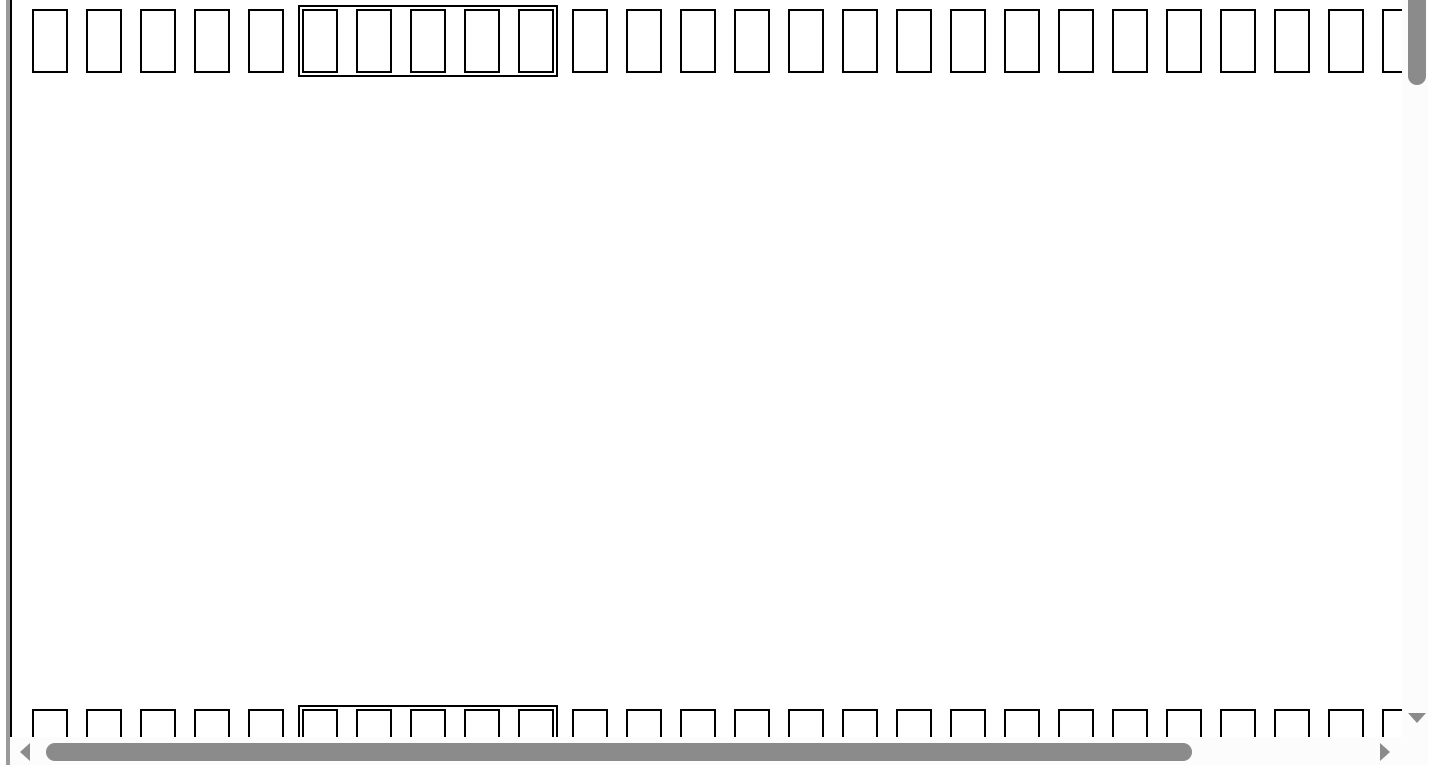
Pentru a avea o implementare care are un throughput mare, vă recomandăm tehnica de selective repeat ARQ. Următoarea simulare prezintă această tehnică. Dacă dați clic pe un segment, acesta va fi pierdut.

Selective Repeat

legend

- ☐ no data received yet
- ☐ data buffered (ready to send, delivered or sent but no ack received yet)
- ☐ ack
- ☐ transmission confirmed
- ☐ data has been delivered to upper network layer

Start Simulation



În temă, vom folosi un timer per conexiune, nu per segment.

API Disponibil

Vom folosi API-ul de sockets UDP pentru a trimite segmentele. Următorul API este opțional și este prezent în scheletul de cod al temei.

Puteți folosi orice structură de date din biblioteca standard de C++ precum și din alte surse, atât timp cât menționați acest lucru în README.

Protocol Transmission Block

Cum protocolul pe care îl dezvoltăm va lucra cu mai multe conexiuni, vom avea nevoie de o tabelă, numită **Protocol Transmission Block**, în care să salvăm mai multe informații despre o anumită conexiune. Vă punem la dispoziție bazele unei astfel de tabelă în structura `struct connection`.

Fiecare conexiune va avea un socket atașat, pe care se va face comunicarea.

```
struct connection {  
    /* TODO: A window for the sender and receiver. */  
    int sockfd; /* UDP socket used for this connection */  
    int conn_id; /* connection identifier */  
    struct sockaddr_in servaddr; /* used to identify the destination */  
    pthread_mutex_t con_lock; /* Used for synchronization with the handler thread  
and read/send calls.*/  
  
    /* TODO. Parameters used only by the sender */  
    int max_window_seq; /* Used to store the max number of packets that can be  
inflight updated  
based on BDP */  
  
    /* TODO. Parameters used only by the client */  
};
```

Vom avea un maxim de 32 de conexiuni. Conexiunile sunt stocate într-un map: `std::map<int, struct connection *> cons;`, unde cheia este un id de conexiune.

Implementare timer per conexiune

Scheletul va pune la dispoziție o funcție de receive care verifică pe toți socketii existenți dacă există date, sau dacă a expirat un timer atașat unui socket.

```
/* Verifica pentru toate conexiunile dacă fie a expirat un timer, fie există  
 * date. Datele sunt scrise în buff, și id-ul conexiunii este scris în conn_id */  
int recv_message_or_timeout(char *buff, size_t len, int *conn_id);
```

Timerele sunt implementate folosind `timerfd`. În `setup_connection` și `wait4connect` este implementată deja funcționalitatea de bază care alocă un socket pentru conexiune și inițializează timer-ul la o secundă. Timer-ul este per conexiune, nu per segment. Puteți alege orice valoare

Thread separat pentru comunicație

În cadrul temei vom folosi două thread-uri, unul principal care aparține aplicației care folosește implementarea protocolului și un al doilea thread dedicat implementării protocolului. Practic, protocolul trebuie să funcționeze în fundal independent de faptul că utilizatorul cheamă funcțiile `send_data` / `recv_data`.

Astfel, în schelet găsiți `recv_handler` și `send_handler` care rulează pe thread-uri separate și sunt pornite din funcțiile de `init`. Aici va fi implementată logica protocolului care se ocupă de transmiterea datelor. Vom folosi `send_data` pentru a primi date de la aplicația client și a le trimite din `send_handler`, pe când `recv_data` va scrie la utilizator în buffer datele din segmentele care au fost primite.

Codul este deja sincronizat folosind mutex-ul `con_lock`. Acest lucru ne permite să avem o implementare independentă de când clientul/serverul cheamă `send_data` / `recv_data`.

Cerințe temă

În [acest repo](#) găsiți scheletul temei, infrastructura și checker-ul automat. Rezolvarea se face în `libsend.cpp` și `librecv.cpp`. Pentru a înțelege modul în care este testată implementarea protocolului, este necesar să consultați fișierele `server.cpp` și `client.cpp`.

Punctajul este împărțit în mai multe componente, după cum urmează:

- **Cărți în format text. (30p).** Protocolul transmite corect cărțile în format text.
- **Imagini (35p).** Protocolul transmite corect imagini.
- **Conexiuni multiple (35p).** Stabilirea conexiunii folosind three way handshake și funcționarea cu mai multe conexiuni în același timp.

Testarea se face prin transmiterea unui fișier între un client și un server. Corectitudinea implementării este dată de lipsa diferențelor între fișierul trimis și cel recepționat. Pentru a simplifica implementarea, serverul doar primește date și răspunde cu segmente de control, în timp ce clientul doar trimite segmente cu date.

Soluția voastră trebuie să funcționeze pentru combinații arbitrare de rate de pierdere de pachete, latențe și viteze ale legăturii de date.

Pentru a primi punctaj pe tema, protocolul trebuie implementat peste UDP folosind API-ul de sockets și să folosească link-ul într-un mod optim, ceea ce înseamnă o fereastră atât la sender, cât și la receiver. În fișierul README trebuie să documentați protocolul dezvoltat.

În realizarea protocolului, se vor implementa următoarele servicii care vor fi folosite de către aplicațiile client și server:

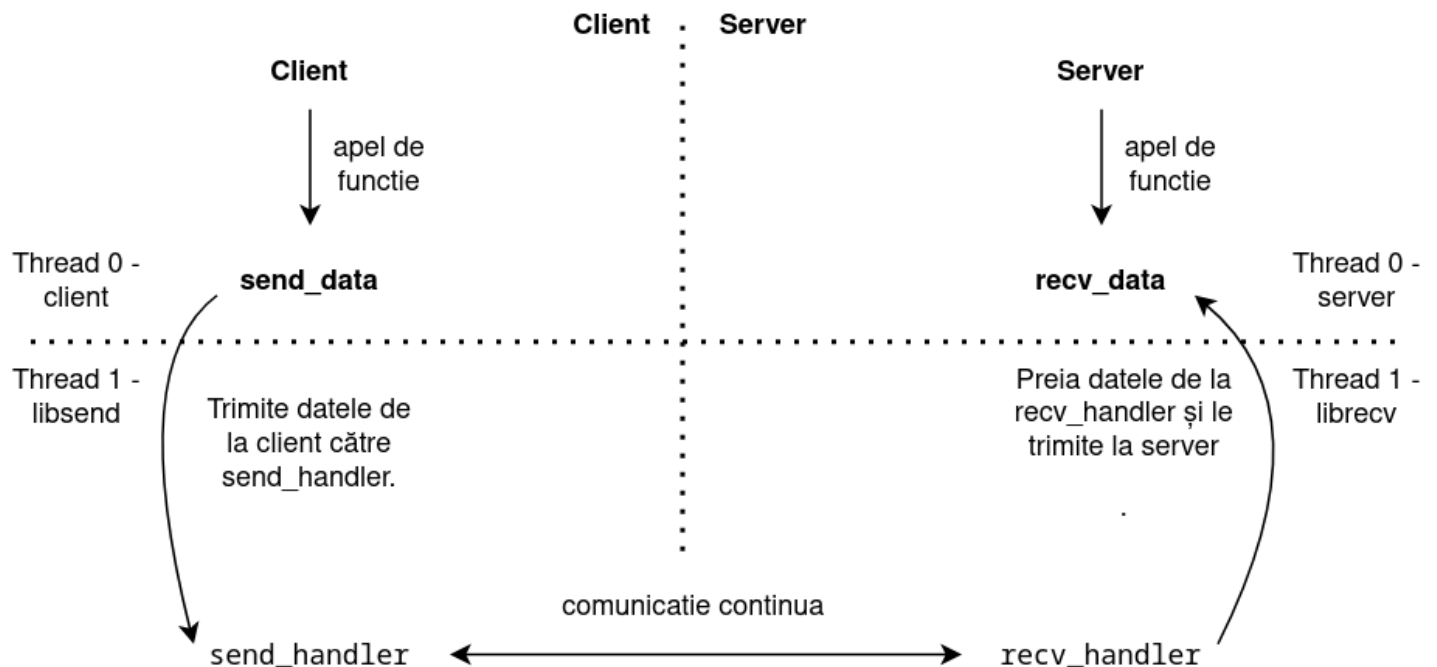

```

/* Folosit pentru a initializa receptorul din protocolul nostru. recv_buffer_bytes
reprezinta
    numarul maxim de bytes ce pot fi stocati de catre receptor pana primeste un
recv_data. */
void init_receiver(int recv_buffer_bytes);
/* Echivalentul lui listent + accept. Este folosit de client pentru a accepta
conexiuni de la
    un client. Returneaza un id de conexiun. Functia se blocheaza pana la
stabilirea
    unei conexiuni. */
int wait4connect(uint32_t ip, uint16_t port);
/* Echivalentul lui recv. Se blocheaza pana are date de scris in buffet */
int recv_data(int connectionid, char *buffer, int len);

/* Folosit pentru a initializa transmitatorul din protocolul nostru.
    Primeste speed si delay pentru a calcula fereastra optima de transmisie */
void init_sender(int speed, int delay);
/* Echivalentul lui connect. Folosit de client pentru a stabili o conexiune. */
int setup_connection(uint32_t ip, uint16_t port);
/* Echivalentul lui send. Nu se blocheaza si returneaza -1 daca nu
    mai are spatiu in buffer-ul intern de transmisie */
int send_data(int conn_id, char *buffer, int len);

```

Interacțiunea dintre componentele temei poate fi prezentată în următoarea diagramă:



Testare

Pentru simularea rețelei vom folosi Mininet. Testarea va presupune transmiterea unui fișier în diferite situații folosind API-ul implementat. Codul aplicațiilor care folosesc implementarea protocolului vostru se găsește în `server.cpp` și `client.cpp`.

```

      8 Mb/s, 1 ms, 5% loss      8 Mb/s, 1 ms, 5% loss
client <-----> router <-----> server
(h1)                (r0)                (h2)

```

Pe h2 vom rula server-ul care primește fișierul și îl scrie în `file.out`:

```
h2# ./server
```

Pe h1 vom rula clientul care trimite un fișier primit ca argument:

```
h1# ./client file_to_send.data
```

Vom testa corectitudinea implementării cu `diff` între cele două fișiere.

Pentru a testa implementarea cu mai multe conexiuni, trebuie să rulați server-ul cu comanda `./server num_connections`, iar apoi să porniți simultan mai mulți clienți de pe h1.

În `checker/test/test_files` găsiți o serie de fișiere care sunt folosite de către checker.

Testare automată

Înainte de a folosi testele automate, vă recomandăm să folosiți modul interactiv al temei pentru a vă verifica corectitudinea implementării. Testarea automată durează câteva minute, așa că este mult mai rapid să testați manual.

Vă punem la dispoziție și o suită de teste, pe care o puteți rula cu argumentul "tests":

```
$ ./checker/checker.sh
```

Pentru fiecare test o să fie create 2 foldere, pentru client și server, în care găsiți loguri și fișierul trimis și recepționat. Checker-ul va afișa aceste două directoare conform snippet-ului de mai

jos. Mai mult, checker-ul va afișa timpul de `cutoff` după care se face verificarea transmisiei fișierelor și timpul efectiv petrecut pentru a transmite fișierul. Pot apărea diferențe față de timpul estimat pe baza parametrilor link-ului deoarece avem overhead introdus de router, scrierea în fișiere și, cel mai mult, de infrastructura Mininet.

```
## Running Test - Corruption Binary - cutoff transfer time 20s - datacenter.jpg
.....
[INFO] Using /tmp/tmp.FzjFRf3dFZ and /tmp/tmp.CGEELKn4tr as tmpdirs
[Info] Running diff -q /tmp/tmp.FzjFRf3dFZ/datacenter.jpg
/tmp/tmp.CGEELKn4tr/file.out
[Info] Total time = 5.967475 seconds

> sudo ls /tmp/tmp.CGEELKn4tr
file.out  server  server.err  server.out
```

Pentru a rula manual un test puteți folosi:

```
$ sudo python3 checker/topo.py test corruption_text
```

În `topo.py`, funcția `run` se ocupă cu testarea automată. De aici puteți modifica parametrii, precum timpul de cutoff per test.

Teste

```
* corruption_text - trimite iterativ trei carti si verifica ca au ajuns ok la
destinatie
* corruption_image - trimite iterativ trei imagini si verifica ca au ajuns ok la
destinatie
* connection_and_multiconnection - porneste mai multi clienti cu fisiere diferite
si verifica ca transferul a fost cu success.
```

Trimiterere

Pentru a fi notată în catalog, tema va fi trimisă pe Moodle, unde checker-ul va fi rulat automat și va pune la feedback nota și output-ul rulării. O rulare completă durează cam 6 minute. Se vor trimite doar arhive generate folosind scriptul `create_archive.sh` din schelet. Vă rugăm să

Trimiteți tema pe Moodle doar după ce ați verificat că acesta rulează bine pe local